

Good. Repos + DataAccess + higher layers are all green. That's the core of E1.

Let's treat that as E1-engineering done and move to the **E1 completion doc** as the next single task.

Task 10 – E1 Completion Doc (drop-in markdown for docs/phase_e1/)

Here's a self-contained markdown doc you can save as something like docs/phase_e1/PhaseE1_Completion.md and later export to PDF if you want.

```
# Phase E1 - Repository & DataAccess Hardening (Completion Document)

## 1. Objective

Phase E1 ensures that Slice's persistence layer and read-side data access are:

- Implemented against the actual Postgres schema.
- Covered by DB-backed integration tests (not just fakes).
- Compatible with the higher-level DataAccess abstraction and its existing unit tests (Phase 8/9).
- Safe to use for downstream context builders and the Streamlit UI.

This phase is non-LLM and non-quant; it is purely about hardening the infrastructure around:

- `Thesis`, `Observation`, `Trade` repositories.
- `DataAccess` orchestration layer.
- Basic portfolio and macro gateways used by E4/E6 later.

---

## 2. Code Changes

### 2.1 ThesisRepository (`src/slice/repositories/thesis_repo.py`)

Status: Implemented and DB-tested.

Key behavior:
```

- Uses ``get_engine()`` and raw SQL (``sqlalchemy.text()``) to talk to the ``thesis`` table.
- Serializes JSON and enums correctly.
- Provides the read APIs required by ``DataAccess``.

Implemented methods:

1. ``insert(self, thesis: Thesis) -> Thesis``

- Upsert into ``thesis`` table:

- JSONB fields serialized via ``json.dumps``:
 - ``drivers``, ``disconfirmers``, ``tags``, ``monitor_indices``
- Enum normalized:
 - ``status`` stored as ``status.value``.
- ``expression`` (list of ``ThesisExpressionLeg``) serialized to JSON array of dicts:
 - ``{"asset", "direction", "size_pct"}``.

- ``ON CONFLICT (id) DO UPDATE`` for idempotent upsert.

2. ``_row_to_thesis(self, row: Mapping[str, Any]) -> Thesis``

- Converts raw DB row $\rightarrow$ ``Thesis`` model.

- JSON decode of:
 - ``drivers``, ``disconfirmers``, ``tags``, ``monitor_indices``, ``expression``
- Normalizes date-like fields into strings:
 - For ``start_date``, ``review_date``:
 - If not ``str`` (e.g. ``date`/`datetime``) $\rightarrow$ ``isoformat()`` $\rightarrow$ string.
 - Fallback to ``str(value)`` if necessary.
- Constructs ``Thesis(**data)``.

3. ``get_by_id(self, thesis_id: str) -> Optional[Thesis]``

- Single-row ``SELECT * FROM thesis WHERE id = :tid``.
- Returns ``None`` if no row.
- Uses ``_row_to_thesis`` for normalization.

4. ``list_all(self) -> List[Thesis]``

- ``SELECT * FROM thesis ORDER BY id ASC``.
- Returns ``List[Thesis]`` via ``_row_to_thesis``.

5. ``list_recent(self, limit: int) -> List[Thesis]``

- ``SELECT * FROM thesis ORDER BY start_date DESC NULLS LAST, id DESC LIMIT :lim``.
- Returns ``List[Thesis]``.

2.2 ObservationRepository (``src/slice/repositories/observation_repo.py``)

****Status:** Implemented, DB-tested, compatible with pgvector column.**

Key behavior:

- Writes to `observation` table including `embedding :: vector`.
- Supports both direct `id` lookup and filtering by `thesis\_ref`.
- Provides a `list\_recent` view ordered by timestamp.

Implemented methods:

1. `insert(self, obs: Observation, embedding\_vector: Optional[List[float]] = None) -> Observation`

- `obs.dict()` → `params`.
- `categories`:
 - If `str`: split on comma → strip → list.
 - Else: assume list.
 - Always serialized via `json.dumps` for JSONB.
- `thesis\_ref` is passed through; schema is expected to handle array/JSONB appropriately.
- `embedding`:
 - If `embedding\_vector` is not `None`:
 - Build string: `[0.00000000,0.00000000,...]` with 8 decimal places.
 - Cast in SQL: `CAST(:embedding AS vector)`.
 - If `None`, `embedding` param is `None`.
- `INSERT ... ON CONFLICT (id) DO UPDATE`:
 - Upserts all fields, including `embedding`.

2. `\_row\_to\_observation(self, row: Mapping[str, Any]) -> Observation`

- Converts raw DB row → `Observation` model.
- For `categories`, `thesis\_ref`:
 - If `str`, attempt `json.loads`; otherwise leave as-is.
- For `sentiment`:
 - If `str`, attempt `Sentiment(value)`.
- Returns `Observation(\*\*data)`.

3. `get\_by\_id(self, obs\_id: str) -> Optional[Observation]`

- `SELECT \* FROM observation WHERE id = :oid`.
- Returns `None` if not found.
- Uses `\_row\_to\_observation`.

4. `get(self, obs\_id: str) -> Optional[Observation]`

- Alias to `get\_by\_id`.

5. `list\_for\_thesis(self, thesis\_id: str) -> List[Observation]`

- Uses array/JSONB containment:

```
```sql
SELECT * FROM observation
```

```
WHERE thesis_ref @> :thesis_ref
ORDER BY timestamp DESC
```

- Binds thesis\_ref as [thesis\_id].
- Returns a list of observations in descending timestamp order.

6. list\_recent(self, limit: int) -> List[Observation]

- SELECT \* FROM observation ORDER BY timestamp DESC LIMIT :lim.

---

## 2.3 TradeRepository (

src/slice/repositories/trade\_repo.py

)

**Status: Implemented and DB-tested.**

Key behavior:

- Writes trades to trade table.
- Lists all trades and trades filtered by thesis\_ref.

Implemented methods:

1. insert(self, trade: Trade) -> Trade

- trade.dict() → params.
- INSERT INTO trade (...) VALUES (...) ON CONFLICT (trade\_id) DO UPDATE SET ...
  - Upserts the record by trade\_id.

2. `_row_to_trade(self, row: Mapping[str, Any]) -> Trade`

- Converts raw DB row  $\rightarrow$  Trade model.
  - If type is a str, attempts `TradeType(value)`.
- Returns `Trade(**data)`.

3. `list_all(self) -> List[Trade]`

- `SELECT * FROM trade ORDER BY timestamp ASC, trade_id ASC.`
- Returns all trades as Trade models.

4. `list_by_thesis(self, thesis_id: str) -> List[Trade]`

- `SELECT * FROM trade WHERE thesis_ref = :thesis_id ORDER BY timestamp DESC.`

5. `list_for_thesis(self, thesis_id: str) -> List[Trade]`

- Alias to `list_by_thesis`.

---

## 2.4 DataAccess (

`src/slice/intelligence/context/data_access.py`

)

**Status: Updated to be robust across real repos and fake test repos; Phase 9 tests pass.**

Core principles:

- **No LLM calls.**
- **No mutations:** read-only façade over the DB and context helpers.
- Compatible with:
  - SQL-backed repositories (implemented in this phase).
  - In-memory fake repos used in tests (Phase 8/9).

Key behaviors:

1. `get_thesis(self, thesis_id: int | str) -> Optional[Thesis]`

Resolution order:

- If repo has `get_by_id`, call `get_by_id(thesis_id)` directly and return.
- Else, if repo has `get`, call `get(thesis_id)`.
- Else, if repo exposes `_theses` (used in tests), return `repo._theses.get(thesis_id)`.
- Else, return `None`.

2. `get_all_theses(self) -> List[Thesis]`

Preference logic:

- If repo has `list_all`, call it.
- Else, if repo has `list_recent`, call `list_recent(limit=100)`.
- Else, return `[]`.

3. `get_observations_for_thesis(self, thesis_id: int | str) -> List[Observation]`

- If `obs_repo` implements `list_for_thesis`, use it directly.
- Else, if `obs_repo` implements `list_all`, filter in Python by matching `thesis_ref` list or scalar.
- Else, return `[]`.

4. `get_recent_observations(self, limit: int = 10) -> List[Observation]`

- If `obs_repo` has `list_recent`, call it.
- Else, if `obs_repo` has `list_all`, sort by timestamp descending and slice.
- Else, return `[]`.

5. `get_risk_snapshot(self) -> Optional[Any]`

- Delegates to `slice.risk.interface.get_snapshot()`.

6. `get_current_portfolio(self) -> PortfolioSnapshot`

- `Internal_build_portfolio_snapshot_dict()`:
  - Reads trades via `trade_repo.list_all()` if available.
  - Normalizes each Trade to a position dict:

```
{
 "symbol": t.symbol or t.asset,
 "quantity": t.quantity,
 "price": t.price,
 "thesis_id": t.thesis_ref,
}
```

- - 
  - Calls `build_portfolio_snapshot(raw_positions)` from `portfolio_adapter`.
- Converts the dict snapshot into:
  - List of Position DTOs.
  - PortfolioTotals DTO.
  - Returns `PortfolioSnapshot`.

7. `get_portfolio_depth(self, theses: Iterable[Any]) -> PortfolioDepthSnapshot`

- Uses existing helpers:
  - `compute_concentration(snapshot_dict)`
  - `compute_factor_exposures(snapshot_dict)`
  - `build_exposure_map(theses, snapshot_dict)`
- Normalizes outputs into:
  - `concentration: Dict[str, float]`
  - `factors: Dict[str, float]` (aggregate factor exposures)
  - `thesis_exposures: Dict[str, float]` including "unassigned" if present.
- Returns `PortfolioDepthSnapshot`.

8. `get_macro_snapshot(self) -> Dict[str, Any]`

- Uses `macro_adapter.build_macro_snapshot`.
- Currently feeds an empty `latest_values` dict (stub) but keeps shape correct.

9. `get_regimes(self) -> Dict[str, str]`

- Calls `compute_regimes` on `get_macro_snapshot()`.
- On exception, falls back to:

```
{"growth": "unknown", "inflation": "unknown",
 "liquidity": "unknown", "usd": "unknown"}
```

10. `get_quant_summaries(self) -> Dict[str, Any]`

- Stubbed deterministic empty payload:

```
{
 "strategies": [],
 "scenarios": [],
 "risk_flags": [],
}
```



---

## 3. Tests Added / Updated

### 3.1 DB-backed Thesis repo tests

File: tests/repositories/test\_thesis\_repository\_db.py

Covers:

- insert + get\_by\_id roundtrip.
- Non-existent get\_by\_id returns None.
- list\_all returns all inserted theses.
- list\_recent ordering by start\_date descending.

These tests use:

- Real Postgres engine from get\_engine().
- apply\_phase4\_schema() via tests/conftest.py to ensure schema exists.
- Non-empty drivers, disconfirmers, expression.
- start\_date passed as YYYY-MM-DD strings (consistent with Thesis model).

### 3.2 DB-backed Observation repo tests

File: tests/repositories/test\_observation\_repository\_db.py

Covers:

- insert + get\_by\_id with:

- Valid Sentiment.
  - categories as list.
  - thesis\_ref as list.
  - **pgvector embedding** with correct dimension (1536).
- list\_for\_thesis:
  - Tests array containment logic (observations with matching thesis\_ref).
  - Ensures only relevant obs are returned.
- list\_recent:
  - Tests timestamp descending ordering.

Note: EMBED\_DIM = 1536 constant used to satisfy vector dimension constraint from the schema.

### 3.3 DB-backed Trade repo tests

File: tests/repositories/test\_trade\_repository\_db.py

Covers:

- insert + list\_all:
  - Ensures ordering by timestamp ASC, trade\_id ASC.
  - Checks that fields (asset, quantity, price, type, thesis\_ref) roundtrip.
- list\_by\_thesis:
  - Filters by thesis\_ref.
  - Ensures ordering by timestamp DESC.

Trade type is chosen generically via:

```
DEFAULT_TRADE_TYPE = list(TradeType)[0]
```

to avoid assuming enum variant names.

### 3.4 DataAccess tests (existing Phase 9)

Files:

- tests/phase9/test\_data\_access\_core.py
- tests/phase9/test\_data\_access\_portfolio.py
- tests/phase9/test\_risk\_interface.py

All re-run and pass with the new implementation:

- get\_thesis works with minimal fake repos.
- get\_all\_theses respects list\_all/list\_recent preference behavior.
- Observation helpers behave as expected.
- Portfolio and risk stubs integrate cleanly.

### 3.5 Portfolio tests (existing Phase 8)

Files:

- tests/phase8/test\_portfolio\_adapter.py
- tests/phase8/test\_portfolio\_depth.py

Re-run and green, confirming no regression from the DataAccess changes.

---

## 4. How to Run the E1 Test Set

From repo root:

```
Core repo integration tests
pytest tests/repositories/test_thesis_repository_db.py \
 tests/repositories/test_observation_repository_db.py \
 tests/repositories/test_trade_repository_db.py -q

DataAccess + portfolio + risk layers
pytest \
```

```
tests/phase8/test_portfolio_adapter.py \
tests/phase8/test_portfolio_depth.py \
tests/phase9/test_data_access_core.py \
tests/phase9/test_data_access_portfolio.py \
tests/phase9/test_risk_interface.py -q

(Optional) Full suite sanity check
pytest -q
```

#### Prerequisites:

- SLICE\_DB\_URL pointing at a Postgres instance with permissions to create/alter the Slice schema.
  - apply\_phase4\_schema() defined and callable without arguments (uses get\_engine() internally).
  - pgvector extension installed with embedding column dimension set to 1536.
- 

## 5. E1 Acceptance Criteria Checklist

- ThesisRepository exposes insert, get\_by\_id, list\_all, list\_recent and is DB-tested.
- ObservationRepository exposes insert, get\_by\_id/get, list\_for\_thesis, list\_recent and is DB-tested, including pgvector embedding.
- TradeRepository exposes insert, list\_all, list\_by\_thesis/list\_for\_thesis and is DB-tested.
- DataAccess is wired to support both:
  - SQL-backed repositories implemented above.
  - Existing fake repos in Phase 8/9 tests.
- All relevant tests pass:
  - Repository DB integration tests.
  - Phase 8 portfolio tests.
  - Phase 9 DataAccess + risk tests.
- No direct DB writes for thesis/observation/trade outside of repositories (except legacy data loaders for market/econ data).

**Conclusion:** Phase E1 repository and data-access hardening is complete and stable enough for downstream E2–E4 work.